

INGEGNERIA DEL SOFTWARE 2

Modulo Software Testing

REPORT FINALE - APACHE OPENJPA

Progettazione manuale, generazione automatica, mutation testing,
reliability e refactoring automatico su due classi target

Studente: Ali Shalby
Docente: Guglielmo De Angelis
A.A. 2025/2026

Repository: github.com/Ali-Shalby/openjpa - branch isw2-project
Baseline production: Apache OpenJPA 4.1.1 - tag 4.1.1
Commit della release: 9d2f5f8c35691c3542606bea1d23b916bcbeda8d

Indice

Sintesi	3
1. Obiettivi e disegno sperimentale	3
2. Ambiente, harness e metriche	4
2.1 Riproducibilità e fairness del confronto	4
3. PCEnhancer: test manuali e adequacy	5
3.1 Gap strutturali di PCEnhancer	6
4. PCEnhancer: mutation e suite automatiche	6
4.1 Mutation progression di PCEnhancer	7
5. ListIteratorWrapper: test manuali e mutation	7
5.1 Stato, oracle e stopping rule	8
6. ListIteratorWrapper: Randoop, EvoSuite e LLM	8
6.1 Confronto same-cardinality	9
7. Interazione LLM e controllo dei prompt	9
8. Varianti C1-C4 e refactoring	10
8.1 Qualità post-refactoring e lettura dei delta	11
9. Reliability empirica	12
10. Minacce alla validità e mitigazioni	12
11. Conclusioni	13
Indice delle tabelle	14
Appendici - tabelle, matrici e tracciabilità	14

Sintesi

Il lavoro presenta uno studio sperimentale di software testing su due classi di Apache OpenJPA 4.1.1, PCEnhancer e ListIteratorWrapper. L'esperimento è organizzato in quattro blocchi collegati. Per ciascuna classe viene prima costruita una suite manuale black-box mediante Category Partition, congelata prima di osservare coverage o mutanti. L'adeguatezza viene poi incrementata con test manuali control-flow, usando Line Coverage e Branch Coverage come due metriche primarie; Method Coverage è registrata come misura supplementare. Successivamente PIT guida l'introduzione di test mutation-oriented, mentre Randoop, EvoSuite e un LLM vengono valutati come tecniche automatiche indipendenti a parità di cardinalità con la suite black-box della stessa classe. Infine vengono generate con LLM quattro varianti refactored, C1-C4, che differiscono per il contesto di test fornito al refactoring; sulle varianti compilabili vengono rigenerate nuove suite automatiche e ne vengono confrontate adequacy, chiarezza, manutenibilità e code smell. I risultati mostrano due profili molto diversi. Su PCEnhancer i test manuali aumentano sensibilmente coverage e capacità di uccidere mutanti, ma le suite automatiche restano limitate dalla complessità della classe; inoltre tutte le varianti C1-C4 riducono gli smell ma non compilano, bloccando correttamente la valutazione dinamica post-refactoring. Su ListIteratorWrapper, invece, la suite manuale raggiunge 98.61% di Line Coverage, 87.50% di Branch Coverage e 96.15% di Mutation Score dopo il miglioramento mutation-guided. Le suite automatiche evidenziano un ordine stabile $T_LLM > T_ES > T_RND$ per mutation adequacy, pur con trade-off distinti su naming, scaffolding e Sonar. Le varianti C1-C4 compilano e preservano i 19 test manuali, ma l'aumento del contesto di test durante il refactoring non produce un miglioramento monotono delle suite automatiche generate successivamente.

1. Obiettivi e disegno dell'esperimento

Lo studio applica, su due classi reali appartenenti allo stesso progetto Apache, tecniche manuali e automatiche di generazione dei test, metriche di adequacy, mutation testing e valutazione di reliability. La baseline scelta è Apache OpenJPA 4.1.1, mantenuta immutabile come C0. Le due classi target sono PCEnhancer e ListIteratorWrapper. La scelta produce un campione volutamente eterogeneo: PCEnhancer è una classe ampia, fortemente integrata con metadati, bytecode enhancement e numerosi servizi OpenJPA; ListIteratorWrapper è molto più compatta, ma mantiene stato interno e presenta percorsi osservabili legati a navigazione, caching, reset e operazioni opzionali. Questa differenza consente di verificare come le tecniche si comportino in condizioni di testabilità molto diverse, evitando di ridurre l'esperimento a due casi sostanzialmente equivalenti. Il quadro complessivo delle suite manuali è sintetizzato in Tabella A.1.

Il disegno sperimentale segue una separazione temporale rigorosa tra progettazione e misurazione. La suite black-box T_BB viene derivata da documentazione e contratto pubblico e congelata prima di qualsiasi lettura di JaCoCo, PIT o struttura privata. Solo dopo il freeze è consentita l'analisi white-box necessaria a T_CF . Analogamente, i survivor PIT sono osservati soltanto dopo il congelamento di $T_BB + T_CF$ e vengono usati per progettare T_MT senza modificare retroattivamente i test precedenti. Le suite automatiche T_RND , T_ES e T_LLM sono generate con protocolli fissati a priori e vengono congelate prima di misurare coverage e mutation score. In questo modo i valori ottenuti restano misure di valutazione e non feedback impliciti usati per ottimizzare le suite dopo aver visto i risultati.

Per rendere il confronto tra tecniche automatiche interpretabile è stata adottata una cardinalità comune, specifica per classe e uguale a N della T_BB : $N=30$ per PCEnhancer e $N=12$ per ListIteratorWrapper. Randoop ed EvoSuite usano seed deterministici e regole di arresto definite prima delle misure; quando un seed produce più test del necessario, la selezione è posizionale e non guidata da coverage o mutation testing. Per l'LLM il prompt principale richiede esattamente N scenari e la loro implementazione; eventuali repair sono ammessi soltanto prima del freeze e solo per problemi di compilazione, rendering dell'output o coerenza runtime, mai sulla base di JaCoCo, PIT o risultati delle altre suite.

L'esperimento risponde a quattro domande operative. La prima riguarda quanto una suite manuale specification-based possa essere resa più adeguata aggiungendo test control-flow e mutation-guided. La seconda confronta, a cardinalità uguale, random testing, coverage-guided generation e LLM test generation. La terza stima la reliability osservata delle suite manuali finali sotto un profilo operativo uniforme. La quarta studia l'effetto del contesto di test sul refactoring automatico: C1 riceve il solo contesto Sonar, C2 anche T_BB , C3 T_BB+T_CF e C4 $T_BB+T_CF+T_MT$; le suite automatiche non sono mai fornite come input al refactoring. Dopo il freeze delle varianti, quando possibile, vengono generati nuovi test automatici per ciascuna configurazione e ne vengono confrontati coverage, mutation score, test strength, chiarezza, manutenibilità e smell. La matrice dei contesti informativi C0-C4 è riportata in [Tabella E.1](#).

2. Ambiente, harness e metriche

Il testing è isolato in un harness Maven dedicato sotto `isw2/testing`. L'harness dipende dagli artefatti production di OpenJPA 4.1.1 ma non riutilizza le suite native del progetto come materiale sperimentale. Questa scelta riduce il rischio di attribuire alle tecniche studiate capacità che derivano in realtà da test preesistenti, e rende più trasparente la provenienza di ogni caso di test. La validazione ordinaria viene eseguita su Java 21 e JUnit Jupiter; i test JUnit 4 prodotti dai generatori sono integrati tramite JUnit Vintage quando necessario. EvoSuite 1.2.0 richiede invece un runtime tecnico Zulu JDK 11 e viene perciò eseguito in una pipeline isolata. La CI GitHub Actions conserva questa separazione, consentendo di verificare automaticamente il ramo di test standard e quello EvoSuite.

Le due metriche primarie di adequacy adottate sono Line Coverage e Branch Coverage. La prima misura la quota di linee production esercitate almeno una volta; la seconda misura la quota di esiti dei branch attraversati. Method Coverage viene riportata come indicatore supplementare, particolarmente utile per `ListIteratorWrapper`, ma non sostituisce le due metriche primarie. Tutte le percentuali JaCoCo sono class-focused: il denominatore è la sola classe target e non l'intero modulo OpenJPA. Questa scelta è essenziale soprattutto per PCEnhancer, poiché l'esecuzione coinvolge numerose dipendenze e fixture che altrimenti diluirebbero la misura.

La robustezza degli oracle viene valutata con PIT. Sono registrati KILLED, SURVIVED, NO_COVERAGE e TIMED_OUT. Il Mutation Score è definito come $\text{KILLED} / (\text{KILLED} + \text{SURVIVED})$. Le due misure rispondono a domande diverse: il Mutation Score penalizza anche il codice mutato non raggiunto, mentre la Test Strength valuta la capacità di distinguere i mutanti che sono effettivamente raggiunti dai test. I timeout sono mantenuti come status raw e non vengono riclassificati manualmente come killed. Per i confronti C0-C4 di `ListIteratorWrapper` la popolazione PIT cambia con il sorgente refactored; perciò si confrontano principalmente percentuali e non conteggi assoluti di mutanti uccisi.

La qualità delle suite automatiche viene osservata anche oltre l'adequacy dinamica. La chiarezza è approssimata mediante la quota di nomi descrittivi rispetto a nomi generati/opachi; la manutenibilità considera dimensione della suite, presenza di scaffolding, assertion-like statements e backlog Sonar. Per Sonar vengono conservate le regole e, ove disponibile, le categorie Clean Code osservate. Queste misure non sono fuse in un singolo indice: un test può essere molto efficace nel mutation testing ma verboso, oppure staticamente pulito ma semanticamente debole. Mantenere separate le dimensioni evita conclusioni artificiali.

Ogni fase produce evidence riproducibili: log di esecuzione, CSV/XML JaCoCo, XML/CSV PIT, hash SHA-256 delle suite congelate quando rilevante, seed e command line dei generatori, prompt e risposte LLM, risultati di stability run, export Sonar e documentazione metodologica. La tracciabilità nel repository permette di risalire dal risultato aggregato al singolo artefatto. Il protocollo privilegia quindi non soltanto il valore finale delle metriche, ma la possibilità di dimostrare quando e con quali informazioni una suite è stata progettata, congelata e misurata. La tracciabilità degli artefatti principali è riepilogata in [Tabella H.2](#).

2.1 Controllo della riproducibilità e fairness del confronto

La riproducibilità è stata trattata come una proprietà sperimentale esplicita. Per le suite random e coverage-guided non è sufficiente indicare il nome del tool: il risultato dipende da seed, budget, classpath, runtime e regola di selezione. Per questo Randoop ed EvoSuite conservano command line, seed e output RAW, mentre le suite canoniche sono separate dagli artefatti non selezionati. In particolare, quando il generatore produce più di N test, il criterio non è 'tenere quelli migliori', perché ciò introdurrebbe un feedback implicito basato sulla successiva misurazione. La regola è invece fissata a priori: consumo dei seed in ordine deterministico e selezione posizionale fino a N. Lo stesso principio vale per il pruning EvoSuite. Questo permette di ripetere il processo e di spiegare perché un determinato test appartenga o non appartenga alla suite finale senza ricorrere alle sue performance.

La fairness non significa rendere identiche tecniche che per natura sono diverse. Randoop costruisce regression test a partire da sequenze di chiamate pubbliche; EvoSuite esegue una ricerca guidata da criteri strutturali e genera scaffolding; l'LLM usa il sorgente e una descrizione esplicita del task. Il controllo adottato è quindi sulla cardinalità, sull'assenza di feedback post-freeze e sulla disponibilità delle informazioni. Randoop ed EvoSuite non ricevono test manuali come seed semantico; T_LLM non riceve gli output delle altre suite, JaCoCo, PIT o survivor. In questo modo nessuna tecnica ottiene un vantaggio perché ha accesso ai risultati con cui verrà poi valutata.

Per le misure PIT viene inoltre preservata l'identità del target production all'interno dello stesso confronto. Nelle tre suite automatiche C0 di una classe la popolazione di mutanti deve coincidere, così che le differenze possano essere attribuite ai test e non a una diversa compilazione del production code. Nelle varianti C1-C4, invece, il codice cambia deliberatamente e anche la popolazione PIT può

cambiare: il protocollo registra tale variazione e passa a confronti percentuali. Questa distinzione evita due errori opposti: pretendere l'identità dei mutanti quando il sorgente è differente, oppure confrontare conteggi assoluti come se la popolazione fosse rimasta invariata.

La CI ha un ruolo di controllo, non di generazione dei risultati. Il normale harness Java 21 verifica le suite manuali e le suite compatibili con JUnit Jupiter/Vintage; EvoSuite viene isolata su JDK 11 per incompatibilità tecniche della versione 1.2.0 con bytecode più recente. La separazione impedisce di modificare la baseline di progetto solo per adattarla a uno strumento legacy. L'esperimento conserva quindi due livelli: il runtime principale coerente con OpenJPA e un runtime tecnico circoscritto, usato solo dove necessario e documentato.

Un ultimo gate riguarda la compilabilità delle varianti production. Coverage, mutation testing e reliability presuppongono l'esistenza di bytecode eseguibile. Se una variante non compila, non è corretto sostituire la misura con una precedente build, né generare test testuali che non possano essere validati. Questo principio, apparentemente semplice, diventa determinante nella fase C1-C4 di PCEnhancer e consente di distinguere un vero risultato negativo del refactoring da un dato mancante per errore operativo.

3. PCEnhancer: progettazione manuale e miglioramento dell'adequacy

La prima classe target, `org.apache.openjpa.enhance.PCEnhancer`, costituisce il caso più difficile dell'esperimento. La suite `T_BB` è stata costruita mediante Category Partition a partire dalla OpenJPA User Guide 4.1.1, dalla Javadoc/API pubblica e da un audit `javap -public`. L'ispezione del sorgente è stata ammessa soltanto quando indispensabile per istanziare una categoria già derivata dalla specifica o completare un oracle già definito dal contratto pubblico. Sono stati esclusi esplicitamente i test nativi OpenJPA, coverage, controllo di flusso privato, mutanti PIT, Sonar e varianti future C1-C4. Il risultato è una suite di 30 frame, organizzata in dieci famiglie funzionali: riconoscimento dei nomi delle subclass dinamiche, flag di default, costruttore no-arg, stato del target di enhancement, property-access restrictions, destinazione del bytecode, rappresentazione/cardinalità dei target, validità dell'invocazione, enhancement diretto rispetto alla subclass generata e livello del contratto di enhancement. Le famiglie funzionali della Category Partition sono riepilogate in Tabella B.1.

Le condizioni al contorno di PCEnhancer sono soprattutto confini di stato e configurazione, non intervalli numerici. La `T_BB` include quindi nome generato rispetto a nome ordinario e stringa vuota, presenza/assenza del costruttore no-arg con flag di riparazione, metadata presenti o assenti, classe già `PersistenceCapable`, enforcement delle property restrictions, output directory nulla o esplicita, zero o un target esplicito, e livelli di enhancement contract corrente o immediatamente precedente. `TBB-026` verifica una proprietà di configurazione documentata, `RuntimeUnenhancedClasses`, assegnandole il valore deliberatamente invalido `definitely-invalid` e aspettando `ParseException`. In questo modo l'oracle resta esternamente osservabile e riconducibile al contratto del tool.

La suite congelata `T_BB` esegue 30/30 test con esito PASS. Su PCEnhancer, JaCoCo misura 43.31% di Line Coverage (1169/2699), 30.24% di Branch Coverage (368/1217) e 65.64% di Method Coverage (107/163). I risultati sono coerenti con la natura black-box della suite: una parte sostanziale dell'API pubblica e dei comportamenti documentati viene esercitata, ma una classe di quasi quattromila LOC e con molte combinazioni interne mantiene numerosi percorsi non attraversati. Questo valore costituisce il punto di partenza, non un obiettivo da massimizzare retroattivamente modificando `T_BB`. Il confronto delle metriche strutturali prima e dopo `T_CF` è riportato in [Tabella C.1](#).

Dopo il freeze viene aperta la fase `T_CF`, intenzionalmente white-box. Un gap audit identifica macro-scenari raggiungibili e semanticamente distinti, evitando di aggiungere micro-test solo per inseguire linee residue. Vengono introdotti cinque test: Application Identity con `IdClass` composita; Detached-state Externalization; standard Java Serialization; relationship-valued primary key/derived identity; e optimized `IdClass` constructor copy. La suite cumulativa sale a 35 test, tutti PASS. L'effetto è rilevante: Line Coverage passa a 70.47% (1902/2699), Branch Coverage a 54.89% (668/1217) e Method Coverage a 86.50% (141/163). Il guadagno è quindi di 27.16 punti percentuali sulle linee e 24.65 sui branch, ottenuto con soli cinque scenari strutturali ad alta leva.

La stopping rule di `T_CF` non richiede il 100% di coverage. Una linea residua è considerata un gap solo se è associata a un percorso legittimamente raggiungibile, distinto, con oracle stabile e valore esplicativo. Questa regola evita di trasformare la coverage in un fine autonomo e mantiene la suite manuale limitata e difendibile. In particolare, su PCEnhancer alcuni rami dipendono da combinazioni interne, stati infrastrutturali o condizioni estremamente specifiche: forzarne il raggiungimento avrebbe aumentato molto il costo della suite senza un beneficio proporzionato sulla comprensione del comportamento pubblico.

3.1 Lettura dei gap strutturali di PCEnhancer

La ricostruzione della Category Partition di PCEnhancer evidenzia una difficoltà tipica del testing di componenti infrastrutturali: molte funzionalità pubbliche non corrispondono a un singolo metodo semplice, ma a configurazioni che producono bytecode, classi dinamiche o stati di enhancement. La cardinalità $N=30$ non è stata fissata come target numerico; è il risultato della selezione di frame distinguibili con oracle osservabili. La suite include sia percorsi positivi sia rifiuti espliciti, e tratta come boundary la transizione fra configurazioni ammesse e non ammesse. Questa impostazione rende la T_BB spiegabile a livello di contratto, ma lascia inevitabilmente scoperti numerosi percorsi interni che dipendono dalla struttura dei metadata e dalle trasformazioni ASM.

L'analisi T_CF è quindi costruita per massimizzare il valore informativo di pochi scenari. TCF-001 esercita la generazione dei metodi di object-id per una IdClass composita; TCF-002 porta l'enhancer nel ramo Externalizable con detached state; TCF-003 raggiunge la strumentazione necessaria alla serializzazione Java; TCF-004 usa una primary key relationship-valued e derived identity; TCF-005 attiva OptimizeldCopy e verifica il percorso del costruttore ottimizzato. Questi test non sono scelti perché corrispondono ai cinque blocchi con più linee mancanti in senso puramente quantitativo, ma perché rappresentano macro-comportamenti distinti, legittimi e dotati di oracle.

Il salto di Method Coverage da 65.64% a 86.50% è importante quanto l'aumento di Line e Branch Coverage: dimostra che i cinque scenari aprono interi gruppi di funzionalità che la suite black-box non raggiungeva. Allo stesso tempo il valore finale di Branch Coverage, 54.89%, segnala che PCEnhancer mantiene numerose decisioni interne non convenientemente esercitabili con una suite class-focused limitata. Fermarsi a questo punto non equivale ad accettare una suite debole; significa riconoscere che coverage residua e test utile non sono sinonimi. La successiva mutation analysis serve proprio a stabilire quali gap strutturali abbiano un effetto osservabile sulla robustezza degli oracle.

4. PCEnhancer: mutation testing e suite automatizzate

Il mutation testing viene avviato sulla suite manuale congelata T_BB+T_CF. Con PIT 1.25.8, mutatori DEFAULTS, un thread e target limitato a PCEnhancer, la baseline genera 1700 mutanti: 466 KILLED, 714 SURVIVED, 520 NO_COVERAGE e nessun timeout. Il Mutation Score è 27.41% e la Test Strength 39.49%. La distanza tra coverage strutturale e mutation adequacy è significativa: pur raggiungendo oltre il 70% delle linee, molti cambiamenti artificiali non vengono discriminati dagli oracle esistenti. È precisamente questo il ruolo della fase T_MT: trasformare l'analisi dei survivor in nuovi test orientati a comportamenti runtime specifici, senza alterare le suite già congelate. La progressione mutazionale delle suite manuali è riepilogata in [Tabella C.2](#).

Cinque test mutation-guided vengono aggiunti. TMT-001 esercita le semantiche runtime di Application Identity e i metodi di copia dei key field; TMT-002 verifica il round-trip di Externalization con detached state; TMT-003 esegue un vero round-trip di Java Serialization; TMT-004 osserva le semantiche runtime dei campi PersistenceCapable/StateManager; TMT-005 copre relationship-valued/derived identity. I primi due checkpoint mostrano già l'effetto progressivo: dopo TMT-001 i KILLED salgono a 633 e la Test Strength al 53.46%; dopo TMT-002 si raggiungono 719 KILLED e 60.73%. Il run finale con tutti i cinque T_MT produce 828 KILLED, 355 SURVIVED, 515 NO_COVERAGE e 2 TIMED_OUT, con Mutation Score 48.71% e Test Strength 69.99%. Rispetto alla baseline si ottengono +362 mutanti uccisi, -359 survivor, +21.30 punti di Mutation Score e +30.50 punti di Test Strength. La suite manuale finale contiene 40 test, tutti PASS.

I due timeout residui vengono conservati come tali e i survivor non sono automaticamente etichettati come equivalenti. La stopping rule considera raggiunto un equilibrio ragionevole dopo cinque comportamenti runtime distinti: la Test Strength è cresciuta di oltre trenta punti e i survivor residui sono distribuiti su dettagli interni per i quali nuovi test avrebbero aumentato notevolmente complessità e accoppiamento. La scelta è quindi di fermare l'evoluzione manuale senza inseguire artificialmente un Mutation Score vicino al 100%.

Le tre suite automatiche C0 vengono poi confrontate con N=30. Randoop 4.3.4, configurato per produrre regression test pubblici e senza error-revealing tests, genera 30 casi validi, tutti PASS. Tuttavia raggiunge soltanto 1.96% delle linee e 0.82% dei branch; PIT uccide 2 mutanti su 1700, con Mutation Score 0.12% e Test Strength 40.00%. Il risultato suggerisce che, su una classe con forte dipendenza da setup e metadata specifici, l'esplorazione randomica dell'API pubblica fatica a costruire stati sufficientemente ricchi.

EvoSuite 1.2.0 viene eseguito con criterio LINE:BRANCH, budget 120 secondi per seed, minimizzazione e massimo 30 test. Seed 0 produce 15 test e seed 1 ne produce 18; per mantenere la regola predefinita vengono usati i 15 del primo seed e i primi 15 del secondo, senza selezione basata sulle metriche. La suite 30/30 PASS raggiunge 3.00% di Line Coverage e 1.56% di Branch Coverage;

uccide 10 mutanti, con Mutation Score 0.59% e Test Strength 71.43%. L'ottimizzazione coverage-guided migliora Randoop ma resta lontana dai percorsi complessi esercitati dalla suite manuale.

La suite T_LLM viene generata tramite Microsoft Copilot Web, in browser chat, con un prompt principale che richiede esattamente 30 scenari TLLM-001...TLLM-030, una breve analisi e l'implementazione Java 21/JUnit Jupiter con tracciabilità uno-a-uno. Il contesto include il sorgente production e l'ambiente tecnico, ma esclude test nativi, suite manuali/automatiche precedenti, JaCoCo, PIT, survivor ed evidenze delle varianti. Prima del freeze vengono effettuati soltanto repair tecnici di rendering/completamento dell'output e compatibilità Mockito sui generic. La suite finale passa 30/30 test e 5/5 stability run. Raggiunge 4.41% di Line Coverage e 3.53% di Branch Coverage, uccide 39 mutanti, con Mutation Score 2.29% e Test Strength 86.67%.

A parità di N, quindi, l'ordine di adequacy su PCEnhancer è $T_LLM > T_ES > T_RND$, ma i valori assoluti restano bassi. La lettura corretta non è che il generatore LLM risolva la complessità della classe: semplicemente, nel protocollo adottato costruisce test più mirati a comportamenti osservabili rispetto agli altri due generatori. Sul piano qualitativo T_LLM usa naming descrittivo nel 100% dei test e 458 LOC senza scaffolding separato; Randoop usa 498 LOC totali, nomi completamente opachi e 57 code smell; EvoSuite è staticamente il più pulito con 0 smell, ma richiede 1404 LOC complessive a causa di 892 LOC di scaffolding e mantiene naming opaco. Adequacy e manutenibilità statica producono dunque classifiche diverse. Il confronto quantitativo tra Randoop, EvoSuite e T_LLM sulla baseline C0 è raccolto in [Tabella D.1](#).

4.1 Interpretazione della mutation progression di PCEnhancer

La progressione PIT di PCEnhancer mostra che l'effetto dei T_MT non è distribuito uniformemente. TMT-001 produce da solo +167 KILLED rispetto alla baseline, perché un oracle runtime su Application Identity riesce a distinguere una famiglia ampia di trasformazioni che la sola verifica strutturale non osservava. TMT-002 aggiunge altri +86 KILLED attraverso il round-trip Externalization. TMT-003, TMT-004 e TMT-005 aggiungono complessivamente altri 109 KILLED rispetto al checkpoint TMT-002. Il dato conferma che pochi test ad alto livello possono avere una leva molto maggiore di molti micro-test progettati localmente attorno a singoli mutanti.

Il numero NO_COVERAGE scende soltanto da 520 a 515, mentre i KILLED crescono di 362. Quasi tutto il beneficio deriva quindi dal rafforzamento degli oracle su codice già raggiunto o reso raggiungibile dagli scenari runtime, non da una drastica espansione della superficie strutturale. Questo è uno dei risultati più significativi del caso PCEnhancer: l'incremento di coverage non è l'unico, né necessariamente il principale, meccanismo con cui una suite può diventare più efficace. La Test Strength, che passa dal 39.49% al 69.99%, rende visibile proprio questa evoluzione.

Il confronto con le suite automatiche rafforza l'interpretazione. Randoop, EvoSuite e LLM hanno Mutation Score molto bassi perché lasciano la maggior parte dei 1700 mutanti in NO_COVERAGE; quando però raggiungono un mutante, EvoSuite e soprattutto T_LLM mostrano una Test Strength relativamente alta. In altre parole, il problema automatico su PCEnhancer è soprattutto costruire setup e stati capaci di entrare nei percorsi profondi della classe. La suite manuale usa conoscenza della specifica e del dominio di enhancement per costruire tali stati, mentre i generatori automatici rimangono maggiormente confinati alla superficie dell'API. Questa differenza deve essere letta come caratteristica del target, non come difetto assoluto degli strumenti.

5. ListIteratorWrapper: Category Partition, control-flow e mutation testing

La seconda classe, org.apache.openjpa.lib.util.collections.ListIteratorWrapper, offre un ambiente più compatto ma non banale. L'oggetto adatta un Iterator a un'interfaccia di tipo ListIterator, memorizzando elementi già attraversati e gestendo navigazione avanti/indietro, reset e alcune operazioni opzionali. La T_BB viene progettata esclusivamente sul comportamento pubblico descritto dalla Javadoc OpenJPA 4.1.1 e dai contratti di Iterator, ListIterator e ResettableListIterator. Non vengono usati sorgente interno, branch, campi privati, JaCoCo, PIT o test nativi. Le categorie sono input al costruttore, tipo dell'iteratore, cardinalità della sequenza, direzione di navigazione, boundary, reset e operazioni opzionali add/set con plain Iterator. L'inventario dei 12 frame black-box congelati è riportato in [Tabella B.2](#).

La suite congelata contiene 12 frame. Oltre ai casi ordinari, rappresenta esplicitamente input null, sequenza vuota, singolo elemento, next oltre la fine, previous prima dell'inizio, reset dopo avanzamento parziale e reset dopo consumo completo. remove() viene deliberatamente escluso dalla T_BB iniziale perché la documentazione pubblica non fornisce un oracle sufficientemente univoco; risolvere l'ambiguità leggendo l'implementazione avrebbe compromesso la natura specification-based della suite. Questa esclusione è metodologicamente importante: un black-box test non dovrebbe essere costruito per confermare ciò che il codice fa, ma per verificare ciò che il contratto consente di aspettarsi.

I 12 test passano integralmente. La baseline JaCoCo è 52.78% Line (38/72), 45.00% Branch (18/40) e 90.91% Method (10/11). Il fatto che quasi tutti i metodi siano raggiunti ma meno della metà dei branch sia coperta mostra perché Method Coverage da sola sarebbe una misura insufficiente. Dopo il freeze, l'analisi control-flow introduce cinque test: un percorso ListIterator con navigazione/indici/reset, delega di add/set/remove su ListIterator, remove su plain Iterator subito dopo next, remove dopo un passo backward e stato invalido per remove. La suite cumulativa di 17 test passa integralmente e porta Line Coverage a 98.61%, Branch Coverage a 87.50% e Method Coverage a 100%. Il guadagno di Branch Coverage, oltre 42 punti percentuali, conferma che i percorsi interni non erano ben rappresentati dalla sola specifica pubblica. Il confronto pre/post T_CF è incluso in [Tabella C.1](#).

Il mutation testing della suite T_BB+T_CF produce una baseline già molto forte: 52 mutanti, 46 KILLED, 4 SURVIVED e 2 NO_COVERAGE, con Mutation Score 88.46% e Test Strength 92.00%. L'analisi non crea un test per mutante; i gap vengono raggruppati per stato comportamentale e percorso necessario a distinguerli. Sono aggiunti soltanto due T_MT. TMT-001 attraversa in una singola sequenza stati cached e frontier per discriminare predicati di hasNext/hasPrevious; TMT-002 consuma un Iterator, esegue reset e verifica che remove venga rifiutato con IllegalStateException mantenendo invariata la lista sottostante.

Dopo i due test, la suite manuale finale contiene 19 casi e passa 19/19. PIT mantiene la stessa popolazione di 52 mutanti ma porta KILLED a 50, SURVIVED a 2 e NO_COVERAGE a 0. Mutation Score e Test Strength raggiungono entrambi 96.15%. Il delta è +4 KILLED, -2 SURVIVED, -2 NO_COVERAGE, +7.69 punti di Mutation Score e +4.15 punti di Test Strength. I due survivor residui sono documentati come candidati forti a equivalenza o discriminazione non fattibile attraverso un oracle pubblico stabile, senza essere dichiarati formalmente equivalenti. La stopping rule evita quindi un TMT-003 privo di una giustificazione comportamentale convincente. L'evoluzione PIT della suite manuale è riportata in [Tabella C.2](#).

Il confronto tra le due classi è istruttivo. Su ListIteratorWrapper cinque T_CF e due T_MT quasi saturano l'adeguatezza strutturale e mutazionale; su PCEnhancer lo stesso tipo di processo produce miglioramenti importanti ma lascia ampi margini. Non è la tecnica a cambiare, bensì la testabilità del target: dimensione, dipendenze, configurazione e profondità dei percorsi influenzano fortemente il costo necessario per costruire stati osservabili. Per questo il progetto evita di imporre una soglia unica di coverage o mutation score valida per entrambe le classi.

5.1 Stato, oracle e stopping rule di ListIteratorWrapper

ListIteratorWrapper rende particolarmente evidente il valore di distinguere tra reachability e osservabilità. Il wrapper può trovarsi in stati in cui elementi sono già stati letti dall'Iterator sottostante e memorizzati internamente, stati in cui il cursore logico torna indietro su elementi cached, e stati frontier in cui per avanzare è necessario interrogare nuovamente l'Iterator reale. Un test che chiama semplicemente tutti i metodi può ottenere Method Coverage elevata senza attraversare queste transizioni. Per questo la T_CF è costruita attorno a sequenze di operazioni e non a chiamate isolate.

L'ambiguità di remove() nella documentazione è gestita separando i livelli di evidenza. remove() non entra nella T_BB perché un oracle black-box forte non è derivabile senza interpretazione aggiuntiva; diventa invece legittimo in T_CF, dopo l'apertura white-box, per esercitare la delega e gli stati di validità. Infine PIT mostra che alcuni predicati di remove e navigazione restano debolmente discriminati e giustifica TMT-001/TMT-002. Lo stesso metodo può quindi comparire in fasi diverse con motivazioni differenti, senza contraddire il freeze: ciò che cambia non è il passato della T_BB, ma la fonte di informazione ammessa in una fase successiva.

Il fatto che PIT post-T_MT riporti anche 72/72 linee raggiunte non sostituisce la misura JaCoCo canonica post T_CF di 71/72=98.61%. Le due osservazioni appartengono a momenti sperimentali diversi: JaCoCo misura l'adequacy strutturale dopo il control-flow improvement; PIT osserva la classe durante la suite arricchita con test mutation-guided. Conservare questa distinzione evita di retrodatare un miglioramento prodotto da T_MT e rende leggibile l'effetto di ciascuna fase.

I due survivor finali sono un ulteriore esempio di stopping rule ragionata. Uno dei mutanti diventa raggiungibile dopo TMT-001 ma resta SURVIVED: il nuovo test migliora la reachability senza produrre un comportamento pubblico diverso. L'audit lo classifica come forte candidato a equivalenza o discriminazione non fattibile, non come mutante formalmente equivalente. La suite si ferma perché non esiste un nuovo scenario pubblico stabile che giustifichi il costo di un TMT-003, non perché ogni survivor residuo sia stato 'spiegato via' a posteriori.

6. ListIteratorWrapper: confronto tra Randoop, EvoSuite e LLM

Per ListIteratorWrapper le tre suite automatiche vengono confrontate a N=12, pari alla T_BB. Randoop 4.3.4 usa seed deterministici consecutivi, 60 secondi per seed, limite di output 12 e sole chiamate pubbliche; ArrayList è introdotta esclusivamente come producer di Iterator. Seed 0 genera 7

test validi e seed 1 ne genera 9; il protocollo conserva i 7 del primo e i primi 5 del secondo, arrestando la selezione appena raggiunti 12 casi. Tutti passano. La suite raggiunge 58.33% Line, 47.50% Branch e 100% Method; PIT uccide 6 mutanti, ne lascia 19 SURVIVED e 27 NO_COVERAGE, per Mutation Score 11.54% e Test Strength 24.00%. Randoop riesce quindi a toccare l'interfaccia in modo ampio a livello di metodi, ma gli oracle di regressione sono deboli rispetto alle variazioni introdotte dai mutanti.

EvoSuite 1.2.0 usa LINE:BRANCH, budget 120 secondi, minimizzazione, JDK 11 e massimo 12 test. Il seed 0 genera 15 casi tutti validi; vengono congelati i primi 12 in ordine, prima di osservare JaCoCo o PIT. La suite raggiunge 83.33% Line, 82.50% Branch e 81.82% Method. Pur raggiungendo meno metodi di Randoop, uccide 29 mutanti su 52, con Mutation Score 55.77% e Test Strength 65.91%. Il confronto mostra che Method Coverage non è sufficiente a descrivere l'efficacia: Randoop tocca 11/11 metodi ma presenta mutation adequacy molto inferiore; EvoSuite costruisce invece sequenze più favorevoli all'esercizio dei branch e alla discriminazione dei mutanti.

La T_LLM C0 usa un prompt principale che richiede 12 scenari TLLM-001...TLLM-012 e codice Java 21/JUnit Jupiter. Il primo run mantiene 12 test ma evidenzia tre failure e tre errori. Poiché questi problemi emergono prima di qualsiasi misura di adequacy, viene applicata una catena di repair documentata R1 -> R2 -> R3. R1 propone le correzioni ma restituisce output troncato; R2 fissa le decisioni principali, incluso l'uso di un CountingIterator realmente plain Iterator e un oracle sugli indici, ma è ancora troncato; R3 completa il codice in modalità code-only. Nessun repair usa coverage, mutanti o risultati di altre suite. La versione finale passa 12/12, supera 5/5 stability run e viene congelata tramite SHA-256.

Dopo il freeze, T_LLM raggiunge 100% Line Coverage, 92.50% Branch Coverage e 100% Method Coverage. PIT sulla popolazione identica di 52 mutanti produce 47 KILLED, 5 SURVIVED e 0 NO_COVERAGE, con Mutation Score e Test Strength pari a 90.38%. A parità di N, l'ordine sulle metriche dinamiche è quindi $T_LLM > T_ES > T_RND$. Il risultato è specifico di questa classe e del protocollo: non dimostra una superiorità generale degli LLM, ma mostra che in un'API compatta e stateful l'LLM ha costruito sequenze e oracle più discriminanti dei due generatori considerati. Il confronto same-cardinality completo su C0 è sintetizzato in [Tabella D.1](#).

Anche qui la valutazione qualitativa completa il quadro. Randoop ed EvoSuite usano naming opaco, mentre T_LLM mantiene nomi descrittivi al 100%. EvoSuite introduce scaffolding dedicato; T_LLM non richiede scaffolding separato. Nel confronto C0 di qualità statica, Randoop presenta 65 smell, EvoSuite 1 e T_LLM 0. Questi valori rafforzano il vantaggio osservato dell'LLM su ListIteratorWrapper, ma il progetto conserva distinte le dimensioni: una suite leggibile non è automaticamente adeguata e una suite con alta branch coverage non ha necessariamente oracle forti.

6.1 Cosa misura davvero il confronto same-cardinality

La cardinalità uguale rende confrontabile il numero nominale di casi di test, ma non equivale a parità di costo computazionale, effort umano o quantità di codice generato. Questo emerge chiaramente su entrambe le classi. Per PCEnhancer 30 test EvoSuite occupano 1404 LOC complessive per effetto dello scaffolding, mentre T_LLM ne usa 458 e Randoop 498. Su ListIteratorWrapper le suite sono più compatte, ma le differenze restano: Randoop 397 LOC, EvoSuite 286, LLM 344 su C0. Per questo N è solo una dimensione di fairness e viene affiancata da LOC, scaffolding, assertion-like statements e Sonar.

Le tecniche producono inoltre oracle di natura diversa. I regression test di Randoop tendono a cristallizzare valori osservati durante la generazione; EvoSuite ottimizza obiettivi strutturali e sintetizza assertion in funzione delle esecuzioni esplorate; l'LLM costruisce scenari descritti semanticamente nel prompt e può quindi esplicitare precondizioni e intenzioni. Il progetto non assume che un tipo di oracle sia intrinsecamente migliore: la valutazione PIT fornisce un criterio comune per osservare quanto le assertion distinguano trasformazioni del production code. Su ListIteratorWrapper il divario 11.54% vs 55.77% vs 90.38% mostra che, a N costante, la qualità delle sequenze e degli oracle conta molto più della sola quantità di test.

La Method Coverage di ListIteratorWrapper offre un controesempio utile a letture superficiali: Randoop raggiunge 100% dei metodi, EvoSuite soltanto 81.82%, ma EvoSuite uccide quasi cinque volte più mutanti. T_LLM raggiunge invece sia 100% Method sia valori molto più alti di Branch e Mutation Score. Questo rafforza la decisione iniziale di usare Line e Branch come metriche primarie e di trattare Method Coverage come misura supplementare. Una suite può toccare un metodo senza attraversarne stati e decisioni semanticamente rilevanti.

7. Interazione con l'LLM e controllo dei prompt

L'interazione con l'LLM è trattata come parte dell'esperimento e non come semplice supporto alla scrittura del codice. Per le suite T_LLM C0 di entrambe le classi viene usato un prompt principale strutturato: identificazione del target e dell'ambiente, vincolo di cardinalità esatta, richiesta di scenari

numerati, codice completo, tracciabilità uno-a-uno e divieto di usare test o risultati non forniti. Il contesto è intenzionalmente limitato al sorgente production e alle informazioni tecniche necessarie. Questo riduce la contaminazione tra tecniche e rende l'output attribuibile alle informazioni effettivamente disponibili al modello.

Nel progetto compaiono però anche altre funzioni di prompt. I prompt di repair sono usati quando il codice generato non è materialmente utilizzabile prima del freeze: completamento di output troncati, correzioni di rendering o adattamenti di setup/runtime. Il caso più articolato è ListIteratorWrapper, per cui la sequenza R1-R3 è conservata integralmente. I prompt di refactoring C1-C4 hanno una funzione diversa: chiedono al modello di riscrivere la classe production utilizzando un contesto crescente di test manuali. Infine, per le varianti compilabili, ulteriori prompt generano nuove T_LLM indipendenti contro ciascuna Cx. Si tratta quindi di diverse tipologie funzionali di interrogazione - generazione, repair/completamento, refactoring condizionato e test generation post-refactoring - ma non di un A/B test controllato su stili di prompting alternativi. Questa distinzione viene mantenuta per non attribuire all'esperimento una comparazione di prompt che non è stata svolta.

Un principio centrale è che il repair non deve trasformarsi in tuning post-hoc. Per questo le correzioni sono ammesse solo prima del freeze e solo a seguito di errori di compilazione o di esecuzione; appena la suite è valida e stabile, viene congelata e da quel momento JaCoCo/PIT diventano strumenti di sola valutazione. Analogamente, i survivor non vengono reinseriti nel contesto LLM per generare un secondo set ottimizzato. Il protocollo evita così un vantaggio informativo non disponibile a Randoop o EvoSuite.

La stabilità viene misurata ripetendo l'esecuzione della suite congelata. PCEnhancer e ListIteratorWrapper T_LLM C0 ottengono entrambe 5/5 run PASS. Nelle nuove suite LLM di ListIteratorWrapper C1-C4 la stabilità resta 5/5; C1 e C4 richiedono un repair tecnico pre-freeze per assicurare che il ramo plain Iterator riceva a runtime un oggetto che non implementa ListIterator. Anche in questi casi nessun feedback di coverage o mutation testing viene usato per correggere la suite.

8. Varianti C1-C4 e refactoring automatico

La fase finale applica un protocollo controllato di refactoring automatico assistito da LLM. C0 è la classe originale. C1 viene generata fornendo al modello il sorgente e il contesto Sonar, senza test; C2 aggiunge T_BB; C3 aggiunge T_BB e T_CF; C4 aggiunge anche T_MT. T_RND, T_ES e T_LLM non sono mai incluse tra gli input del refactoring. Ogni variante è ottenuta indipendentemente da C0, congelata e valutata senza iterazioni di correzione basate sui risultati post-hoc. L'obiettivo è osservare se una maggiore disponibilità di test manuali durante il refactoring aiuti a preservare funzionalità e qualità, non addestrare progressivamente il modello sulle risposte precedenti. La matrice esatta degli input forniti al modello per C1-C4 è mostrata in Tabella E.1.

Su PCEnhancer l'esito più importante è negativo ma sperimentalmente informativo: nessuna variante C1-C4 compila. C1 e C2 usano InsnList.isEmpty(), metodo non disponibile nell'API effettiva; C3 introduce una incompatibilità String/String[] nel campo exceptions di MethodNode; C4 lascia potenzialmente non inizializzato il campo final repos. Non essendoci bytecode valido, le suite post-hoc non possono essere eseguite e le nuove suite automatiche runtime non possono essere generate e misurate in modo coerente. Il loro stato è quindi NOT RUN / BLOCKED BY VARIANT COMPILATION, non FAIL. Anche una suite LLM testuale non sarebbe validabile contro la variante e produrrebbe pseudo-delta privi di significato.

L'analisi Sonar di PCEnhancer mostra comunque che tutte le varianti riducono il numero totale di smell rispetto a C0=190: C1 scende a 138 (-52), C2 a 141 (-49), C3 a 162 (-28) e C4 a 167 (-23). Le varianti eliminano issue esistenti ma introducono anche nuove issue: rispettivamente 3, 9, 34 e 39. Nessuna introduce una categoria di regola completamente nuova rispetto alla baseline. La sola riduzione di NSmells non è quindi sufficiente a dichiarare il refactoring riuscito: la compilabilità è un gate preliminare e tutte le Cx aumentano inoltre LOC rispetto a C0 (C1/C2 +28, C3/C4 +3), una feature positivamente correlata con BUGGY nel dataset del progetto. Il caso PCEnhancer mostra perché qualità statica e correttezza costruttiva debbano essere considerate congiuntamente. I risultati completi di compilazione, smell e LOC di PCEnhancer sono riportati in [Tabella F.1](#).

Su ListIteratorWrapper l'esito è opposto. Tutte le varianti compilano e tutte superano 19/19 test della suite manuale finale T_BB+T_CF+T_MT, usata post-hoc esclusivamente per validazione. La baseline C0 ha zero smell. C1 e C3 introducono una issue java:S1125 ciascuna; C2 e C4 restano a zero smell. LOC rimane 134 in C1 e C2, scende a 132 in C3 e sale a 139 in C4. Considerando compilazione, preservazione dei test manuali, smell e feature snapshot positivamente correlate con bugginess, C2 è la condizione più favorevole: compila, passa 19/19, mantiene zero smell e non aumenta né LOC né NSmells. I corrispondenti risultati di compilazione, regressione, smell e LOC sono riportati in [Tabella F.2](#).

Poiché tutte le varianti ListIteratorWrapper sono compilabili, il protocollo post-refactoring può essere completato integralmente. Per ogni Cx vengono generate nuove suite indipendenti T_RND, T_ES e T_LLM con N=12 e freeze pre-adequacy. La popolazione PIT cambia con il bytecode: 52 mutanti in C0, 51 in C1, 55 in C2, 47 in C3 e 53 in C4. Questo impedisce confronti grezzi sui KILLED e rende necessario usare Mutation Score e Test Strength come percentuali. Vengono inoltre misurati coverage, LOC dei test, naming, scaffolding e smell Sonar.

Randoop mantiene una Line Coverage circa 58-62% su tutte le varianti, con Mutation Score che passa dall'11.54% di C0 al massimo 20.00% di C2 e poi scende al 13.21% di C4. C1 è particolarmente sfavorevole alla manutenibilità: la suite cresce a 695 LOC e 154 smell contro 397 LOC e 65 smell in C0. Il naming resta 0% descrittivo ovunque. Non emerge alcuna progressione C1->C4. La matrice completa dei risultati Randoop C0-C4 è riportata in [Tabella G.1](#).

EvoSuite è più sensibile al sorgente refactored. C2 raggiunge 92.54% Line, 93.18% Branch, 100% Method e 67.27% Mutation Score; C4 raggiunge la massima coverage, 94.74% Line e 97.62% Branch, ma Mutation Score 66.04%, leggermente inferiore a C2. C3 torna vicino alla baseline. La dimensione resta stabile tra 286 e 304 LOC e lo scaffolding circa 100 LOC; il naming è sempre opaco. Anche in questo caso più test forniti al refactoring non implicano automaticamente una suite successiva più forte. La matrice completa dei risultati EvoSuite C0-C4 è riportata in [Tabella G.2](#).

T_LLM mantiene il profilo più stabile: 100% Line e 100% Method su C0-C4, Branch tra 92.50% e 97.62%, e Mutation Score sempre superiore all'86%. Il massimo è C1 con 96.08%; C2, C3 e C4 ottengono rispettivamente 89.09%, 87.23% e 86.79%. Tutte le suite hanno 100% naming descrittivo e nessuno scaffolding separato; Sonar rileva da 0 a 1 smell per variante. Un confronto particolarmente utile è C4: EvoSuite ha Branch Coverage 97.62%, maggiore del 95.24% di T_LLM, ma Mutation Score 66.04% contro 86.79%. Il dato dimostra concretamente che una coverage strutturale più alta non garantisce oracle più forti. La matrice completa dei risultati T_LLM C0-C4 è riportata in [Tabella G.3](#).

Il quadro complessivo delle nuove suite automatiche ListIteratorWrapper è quindi stabile per tecnica ma non monotono per configurazione. T_LLM presenta sempre il Mutation Score più alto, T_ES occupa la posizione intermedia e T_RND la più bassa. Le categorie Clean Code osservate nelle analisi Sonar includono INTENTIONAL, CONSISTENT e ADAPTABLE. L'effetto della quantità di test fornita al refactoring dipende dal generatore: Randoop massimizza il mutation score in C2, EvoSuite massimizza mutation score in C2 e coverage in C4, mentre T_LLM massimizza mutation score in C1. Nel perimetro dell'esperimento, la tecnica di generazione successiva incide quindi più della semplice quantità di testing context consegnata al modello durante il refactoring.

8.1 Qualità delle suite post-refactoring e lettura dei delta

Il confronto C0-C4 di ListIteratorWrapper aggiunge una dimensione che non è presente nel semplice confronto delle suite C0: la classe production cambia, quindi cambia anche ciò che è facile o difficile generare automaticamente. Randoop è relativamente stabile su coverage ma molto sensibile alla forma del codice generato in C1, dove la suite cresce fino a 695 LOC e 154 smell. Le regole Sonar osservate includono S1220, S1481, S1854, S2133, S3415, S3577 e S5785, ricadendo nelle categorie INTENTIONAL, CONSISTENT e ADAPTABLE. C2, C3 e C4 tornano invece a un profilo statico vicino alla baseline. Questo mostra che una piccola modifica production può cambiare fortemente il modo in cui un generatore random compone e ripete le sequenze.

EvoSuite presenta il comportamento opposto: la dimensione resta molto stabile, fra 286 e 304 LOC, e lo scaffolding fra circa 100 e 101 LOC, ma l'adequacy cambia sensibilmente. C2 e C4 sono nettamente migliori della baseline, mentre C3 torna quasi ai livelli C0. Sonar è minimo in C1 e cresce da C2 a C4, con regole che toccano attributi CLEAR, CONVENTIONAL, COMPLETE e LOGICAL. Il fatto che C4 produca la massima Branch Coverage ma non il massimo Mutation Score suggerisce che il refactoring modifica la geometria dei branch in modo favorevole alla ricerca, senza necessariamente rendere gli oracle più discriminanti.

T_LLM è meno sensibile in termini di coverage: tutte le varianti conservano 100% Line e Method, con Branch sempre sopra 95% eccetto la baseline a 92.50%. Tuttavia il Mutation Score massimo è C1=96.08%, cioè la variante refactored senza test come input, e diminuisce monotonicamente fino a C4=86.79%, senza seguire un andamento lineare. Questa traiettoria contraddice l'ipotesi intuitiva 'più test forniti al refactoring, migliore testabilità successiva'. I dati non consentono di identificare causalmente il meccanismo che produce tale andamento: mostrano soltanto che il modo in cui il sorgente viene riscritto interagisce con il successivo processo di test generation e può rendere più o meno agevole costruire oracle forti.

Il dato qualitativo T_LLM resta invece molto stabile: 100% naming descrittivo, nessun scaffolding separato, da 289 a 344 LOC e da zero a un solo smell Sonar; la sola regola osservata in C1-C3 è S1598, categoria CONSISTENT e attributo CONVENTIONAL. La stabilità 5/5 è mantenuta su tutte le

varianti. C1 e C4 richiedono un repair pre-freeze sul ramo plain Iterator, ma il repair viene applicato prima delle misure e non altera il protocollo di confronto.

Nel complesso, la matrice C0-C4 non identifica una configurazione universalmente migliore per la generazione automatica. C2 è il punto di massimo Mutation Score per Randoop ed EvoSuite; C4 è il punto di massima coverage EvoSuite; C1 è il punto di massimo Mutation Score LLM. Per la classe production, invece, C2 è il refactoring più favorevole rispetto a compilazione, regressione manuale, smell e LOC. Questa differenza fra 'miglior production variant' e 'variante che facilita di più un dato generatore' è uno dei risultati più rilevanti: la testabilità automatica è una proprietà emergente dell'interazione fra codice e tecnica, non un attributo deducibile da una sola metrica statica.

9. Reliability empirica

La reliability viene stimata sulla suite manuale finale congelata di ciascuna classe, assumendo un profilo operativo uniforme sull'insieme dei test. In tale modello ogni test rappresenta un evento operativo equiprobabile. Per PCEnhancer $N=40$ e la probabilità associata a ciascun test è $1/40=0.025$; tutti i 40 test passano, quindi la reliability empirica osservata è $R_{\hat{}}=1.000000$ e la probabilità di failure $Q_{\hat{}}=0.000000$. Per ListIteratorWrapper $N=19$, probabilità per test $1/19=0.052631578947$, e 19/19 test passano, producendo la stessa stima $R_{\hat{}}=1.000000$ e $Q_{\hat{}}=0.000000$. I valori numerici delle due stime sono riepilogati in [Tabella H.1](#).

Questi valori non vengono interpretati come prova di affidabilità perfetta del software in produzione. La stima è condizionata al profilo operativo finito costruito dall'esperimento e alla qualità degli oracle della suite manuale. Un utilizzo reale potrebbe assegnare probabilità molto diverse ai comportamenti o includere stati non rappresentati. Il valore 1.0 indica quindi soltanto che, nel profilo uniforme esplicitamente assunto, nessuno dei casi selezionati osserva una failure della baseline. L'esplicitazione di tale limite è parte integrante della misura.

10. Minacce alla validità e mitigazioni

Validità di costrutto. Coverage, mutation score, naming e smell sono proxy di qualità, non la qualità stessa. La Line/Branch Coverage misura esecuzione strutturale ma non forza oracle significativi; il mutation testing dipende dai mutatori scelti e può includere mutanti equivalenti o difficili da discriminare; i code smell Sonar misurano maintainability statica e non comportamento. La mitigazione consiste nell'uso congiunto di più dimensioni e nel rifiuto di un indice aggregato unico. Il confronto C4 di ListIteratorWrapper tra branch coverage e mutation score evidenzia esplicitamente questa necessità.

Validità interna. Le suite potrebbero essere influenzate da informazioni osservate in fasi precedenti. Per ridurre questo rischio il progetto adotta freeze sequenziali: T_BB prima di coverage/PIT, T_CF prima del mutation-guided improvement, T_RND/T_ES/T_LLM prima delle rispettive misure. Gli eventuali repair LLM sono permessi soltanto per problemi pre-freeze di validità tecnica e sono documentati. La selezione di test automatici eccedenti N è posizionale, non basata sulle performance. Le suite automatiche non sono fornite al refactoring C1-C4, evitando contaminazione tra generazione dei test e generazione del production code.

Validità esterna. Due classi dello stesso progetto non consentono generalizzazioni statistiche sull'efficacia relativa di Randoop, EvoSuite o LLM. Anzi, i due target mostrano comportamenti molto diversi: su PCEnhancer tutte le tecniche automatiche C0 hanno bassa coverage assoluta, mentre su ListIteratorWrapper l'LLM raggiunge una adequacy molto elevata. Il report limita quindi le conclusioni ai target, versioni, tool e protocolli adottati e interpreta i risultati come casi di studio replicabili, non come ranking universale dei generatori.

Validità di conclusione. Le popolazioni PIT C0-C4 di ListIteratorWrapper non sono identiche perché il refactoring cambia il bytecode; per questo il confronto usa percentuali e non KILLED assoluti. Per PCEnhancer la mancata compilazione impedisce qualsiasi misura dinamica sulle varianti: riportare valori C1-C4 avrebbe significato usare bytecode stale o test non validabili. La scelta BLOCKED preserva la correttezza del confronto anche se rende incompleta la matrice numerica. Analogamente, i due TIMED_OUT del PIT finale PCEnhancer restano timeout raw e non vengono conteggiati come killed.

Riproducibilità. Randoop ed EvoSuite sono sensibili a seed, runtime e configurazione; questi parametri sono registrati e la stopping rule è deterministica. L'LLM resta intrinsecamente non deterministico e dipende dal modello disponibile. La mitigazione consiste nella conservazione di prompt, risposte, sorgenti congelati, hash e stability run. Per PCEnhancer C1 la risposta completa della conversazione Copilot originaria non è più disponibile; non è stata ricostruita o rigenerata artificialmente. Restano preservati il sorgente congelato, gli input e l'evidenza di compilazione. Questa lacuna riduce la riproducibilità di quella specifica generazione ma non modifica l'esito osservato sul sorgente salvato.

Validità della specifica. La Category Partition dipende dalla qualità della documentazione pubblica. In ListIteratorWrapper la semantica documentata di remove() è stata giudicata non sufficientemente univoca e il caso è stato escluso dalla T_BB, invece di ricavare l'oracle dal codice production. In PCEnhancer, dove la specifica è più distribuita tra User Guide, Javadoc e API, è stato imposto un ordine di priorità delle fonti e l'ispezione del sorgente è stata limitata. Queste decisioni riducono il rischio di white-box leakage ma non possono eliminare totalmente ambiguità o incompienze della documentazione.

Minaccia legata ai prompt. Il progetto sperimenta più ruoli di interrogazione LLM - generazione, repair/completamento, refactoring con contesto differenziato e test generation post-refactoring - ma non esegue un confronto controllato fra molte formulazioni alternative del prompt principale. Non viene quindi attribuita causalità a una specifica scelta linguistica. La variabile sperimentale controllata nella fase C1-C4 è soprattutto il contenuto di test disponibile al refactoring, non lo stile retorico del prompt.

11. Conclusioni

L'esperimento mostra che l'adeguatezza dei test emerge dall'interazione tra tecnica e testabilità del target. La Category Partition produce una base specification-based interpretabile; l'analisi control-flow consente di colmare gap strutturali senza contaminare la progettazione black-box; il mutation testing rivela debolezze degli oracle che la sola coverage non rende visibili. Su PCEnhancer cinque T_CF elevano Line Coverage dal 43.31% al 70.47% e Branch dal 30.24% al 54.89%, mentre cinque T_MT portano il Mutation Score dal 27.41% al 48.71% e la Test Strength dal 39.49% al 69.99%. Su ListIteratorWrapper cinque T_CF portano Line/Branch al 98.61%/87.50% e due T_MT portano Mutation Score/Test Strength al 96.15%, mantenendo la suite manuale finale a soli 19 test.

Il confronto automatico a cardinalità uguale mostra un pattern coerente nei due target, pur con scale molto diverse: T_LLM ottiene la miglior mutation adequacy, EvoSuite si colloca al centro e Randoop in fondo. Tuttavia la qualità non è monodimensionale. Su PCEnhancer EvoSuite produce zero smell ma richiede molto scaffolding e presenta naming opaco; T_LLM è più compatta e leggibile. Su ListIteratorWrapper T_LLM combina adequacy elevata, naming descrittivo e backlog Sonar minimo, ma il risultato non viene generalizzato oltre il caso di studio.

La fase C1-C4 evidenzia infine che fornire più test al refactoring non garantisce un miglioramento progressivo. PCEnhancer riduce gli smell in tutte le varianti ma perde la compilabilità, mostrando che un refactoring staticamente più pulito può essere inutilizzabile. ListIteratorWrapper preserva la funzionalità in tutte le varianti, ma le nuove suite automatiche non migliorano monotonicamente con il contesto: C2 è favorevole a Randoop ed EvoSuite sul mutation score, C4 massimizza la coverage EvoSuite, mentre C1 massimizza la mutation adequacy LLM. Il risultato principale per la pratica è quindi metodologico: valutare un refactoring e una suite di test richiede gate di compilazione, oracoli dinamici, metriche strutturali, mutation analysis e qualità/manutenibilità dei test; nessuna singola metrica è sufficiente.

Il repository conserva i protocolli, i sorgenti dei test, i prompt, le evidenze di esecuzione e i confronti finali, rendendo il caso di studio verificabile e ripetibile nei limiti degli strumenti non deterministici.

Indice delle tabelle

Le voci seguenti riportano la pagina delle tabelle nelle appendici; i richiami nel testo principale utilizzano la stessa numerazione.

Tabella A.1	- Suite manuali e ruolo sperimentale.....	14
Tabella B.1	- Inventory sintetico della Category Partition di PCEnhancer.....	14
Tabella B.2	- Inventory T_BB congelato di ListIteratorWrapper.....	15
Tabella C.1	- Coverage JaCoCo delle suite manuali prima e dopo T_CF.....	15
Tabella C.2	- Evoluzione della mutation adequacy delle suite manuali.....	15
Tabella D.1	- Same-cardinality comparison delle suite automatiche sulla baseline C0.....	15
Tabella E.1	- Informazioni fornite al modello durante il refactoring. Le suite automatiche non sono input.....	15
Tabella F.1	- PCEnhancer C0-C4: compilazione, smell e LOC.....	16
Tabella F.2	- ListIteratorWrapper C0-C4: compilazione, regressione manuale, smell e LOC.....	16
Tabella G.1	- Randoop: risultati C0-C4 delle nuove suite automatiche (N=12).....	17
Tabella G.2	- EvoSuite: risultati C0-C4 delle nuove suite automatiche (N=12).....	17
Tabella G.3	- LLM: risultati C0-C4 delle nuove suite automatiche (N=12).....	17
Tabella H.1	- Reliability empirica sotto profilo operativo uniforme.....	17
Tabella H.2	- Mappa sintetica degli artefatti del repository.....	17

Appendici - tabelle, matrici e tracciabilità

Le appendici raccolgono le tabelle quantitative richiamate nel testo principale. La discussione e l'interpretazione dei risultati sono state mantenute nel corpo del report; le tabelle servono come supporto verificabile e come collegamento sintetico alle evidenze del repository.

Appendice A - Protocollo e suite manuali

Classe	Suite	Tecnica	N	Esito	Funzione nel protocollo
PCEnhancer	T_BB	Category Partition black-box	30	30/30 PASS	Baseline specification-based
PCEnhancer	T_CF	Control-flow manuale	5	35/35 cumulativi PASS	Adequacy improvement
PCEnhancer	T_MT	Mutation-guided manuale	5	40/40 cumulativi PASS	Oracle robustness
ListIteratorWrapper	T_BB	Category Partition black-box	12	12/12 PASS	Baseline specification-based
ListIteratorWrapper	T_CF	Control-flow manuale	5	17/17 cumulativi PASS	Adequacy improvement
ListIteratorWrapper	T_MT	Mutation-guided manuale	2	19/19 cumulativi PASS	Oracle robustness

Tabella A.1 - Suite manuali e ruolo sperimentale.

Appendice B - Category Partition

Famiglia PCEnhancer	Funzionalità	Frame
F1	Dynamic persistence-capable subclass names	TBB-001..004, 006..007
F2	Default PCEnhancer.Flags	TBB-005
F3	Required no-argument constructor	TBB-008..011
F4	Enhancement target state and outcome	TBB-012..015
F5	Property-access restriction enforcement	TBB-016..018
F6	Bytecode recording destination	TBB-019..020
F7	Target representation and cardinality	TBB-021..025
F8	Tool invocation validity	TBB-026
F9	Direct enhancement vs generated subclass	TBB-027..028
F10	Enhancement contract level	TBB-029..030

Tabella B.1 - Inventory sintetico della Category Partition di PCEnhancer.

ID LIW	Scenario	Oracle
TBB-001	costruttore con null	NullPointerException
TBB-002	plain Iterator vuoto	no next/previous; indici 0/-1

ID LIW	Scenario	Oracle
TBB-003	next() su iteratore vuoto	NoSuchElementException
TBB-004	previous() in posizione iniziale	NoSuchElementException
TBB-005	un elemento, forward/backward	stesso elemento e indici coerenti
TBB-006	navigazione forward completa	A,B,C
TBB-007	navigazione backward completa	C,B,A
TBB-008	next -> previous -> next	A,A,A
TBB-009	reset dopo avanzamento parziale	ritorno alla posizione iniziale
TBB-010	reset dopo consumo completo	seconda iterazione A,B,C
TBB-011	add() su plain Iterator	UnsupportedOperationException
TBB-012	set() su plain Iterator	UnsupportedOperationException

Tabella B.2 - Inventory T_BB congelato di ListIteratorWrapper.

Appendice C - Adequacy manuale e mutation testing

Classe / Stage	Test	Line	Branch	Method
PCEnhancer T_BB	30	43.31%	30.24%	65.64%
PCEnhancer T_BB+T_CF	35	70.47%	54.89%	86.50%
ListIteratorWrapper T_BB	12	52.78%	45.00%	90.91%
ListIteratorWrapper T_BB+T_CF	17	98.61%	87.50%	100.00%

Tabella C.1 - Coverage JaCoCo delle suite manuali prima e dopo T_CF.

Classe / Stage	KILLED	SURVIVED	NO_COV	TIMEOUT	Mutation	Strength
PCEnhancer baseline 35	466	714	520	0	27.41%	39.49%
PCEnhancer final 40	828	355	515	2	48.71%	69.99%
LIW baseline 17	46	4	2	0	88.46%	92.00%
LIW final 19	50	2	0	0	96.15%	96.15%

Tabella C.2 - Evoluzione della mutation adequacy delle suite manuali.

Appendice D - Confronto automatico su C0

Classe	Tecnica	N	Line	Branch	Method	Mut.Score	Strength	LOC	Sonar
PCEnhancer	RND	30	1.96%	0.82%	-	0.12%	40.00%	498	57
PCEnhancer	ES	30	3.00%	1.56%	-	0.59%	71.43%	1404	0
PCEnhancer	LLM	30	4.41%	3.53%	-	2.29%	86.67%	458	6
LIW	RND	12	58.33%	47.50%	100%	11.54%	24.00%	397	65
LIW	ES	12	83.33%	82.50%	81.82%	55.77%	65.91%	286	1
LIW	LLM	12	100%	92.50%	100%	90.38%	90.38%	344	0

Tabella D.1 - Same-cardinality comparison delle suite automatiche sulla baseline C0.

Appendice E - Matrice di refactoring C0-C4

Variante	Sorgente	Sonar	T_BB	T_CF	T_MT	T_RND	T_ES	T_LLM
C0	Originale	N/A	N/A	N/A	N/A	N/A	N/A	N/A
C1	Refactored	Sì	No	No	No	No	No	No
C2	Refactored	Sì	Sì	No	No	No	No	No
C3	Refactored	Sì	Sì	Sì	No	No	No	No
C4	Refactored	Sì	Sì	Sì	Sì	No	No	No

Tabella E.1 - Informazioni fornite al modello durante il refactoring. Le suite automatiche non sono input.

Appendice F - Esito M4 sulle classi production

PCEnhancer	Compile	NSmells	Delta smell	New issues	LOC	Delta LOC
C0	PASS	190	0	0	3958	0
C1	FAIL	138	-52	3	3986	+28
C2	FAIL	141	-49	9	3986	+28
C3	FAIL	162	-28	34	3961	+3

PCEnhancer	Compile	NSmells	Delta smell	New issues	LOC	Delta LOC
C4	FAIL	167	-23	39	3961	+3

Tabella F.1 - PCEnhancer C0-C4: compilazione, smell e LOC.

ListIteratorWrapper	Compile	Post-hoc	NSmells	LOC	Feature +corr aumentata
C0	PASS	baseline	0	134	-
C1	PASS	19/19 PASS	1	134	NSmells +1
C2	PASS	19/19 PASS	0	134	NO
C3	PASS	19/19 PASS	1	132	NSmells +1
C4	PASS	19/19 PASS	0	139	LOC +5

Tabella F.2 - ListIteratorWrapper C0-C4: compilazione, regressione manuale, smell e LOC.

Appendice G - Suite automatiche ListIteratorWrapper C0-C4

Randoop

Cx	Line	Branch	Method	Mut.Score	Strength	LOC	Sonar	Naming desc.
C0	58.33%	47.50%	100.00%	11.54%	24.00%	397	65	0%
C1	58.82%	40.48%	91.67%	17.65%	32.14%	695	154	0%
C2	58.21%	52.27%	100.00%	20.00%	37.93%	394	69	0%
C3	57.97%	52.38%	100.00%	19.15%	34.62%	394	69	0%
C4	61.84%	50.00%	100.00%	13.21%	26.92%	394	69	0%

Tabella G.1 - Randoop: risultati C0-C4 delle nuove suite automatiche (N=12).

EvoSuite

Cx	Line	Branch	Method	Mut.Score	Strength	LOC	Sonar	Naming desc.
C0	83.33%	82.50%	81.82%	55.77%	65.91%	286	1	0%
C1	88.24%	88.10%	91.67%	64.71%	68.75%	292	0	0%
C2	92.54%	93.18%	100.00%	67.27%	67.27%	300	7	0%
C3	82.61%	80.95%	83.33%	55.32%	63.41%	286	8	0%
C4	94.74%	97.62%	100.00%	66.04%	66.04%	304	10	0%

Tabella G.2 - EvoSuite: risultati C0-C4 delle nuove suite automatiche (N=12).

LLM

Cx	Line	Branch	Method	Mut.Score	Strength	LOC	Sonar	Naming desc.
C0	100.00%	92.50%	100.00%	90.38%	90.38%	344	0	100%
C1	100.00%	97.62%	100.00%	96.08%	96.08%	289	1	100%
C2	100.00%	95.45%	100.00%	89.09%	89.09%	298	1	100%
C3	100.00%	97.62%	100.00%	87.23%	87.23%	324	1	100%
C4	100.00%	95.24%	100.00%	86.79%	86.79%	321	0	100%

Tabella G.3 - LLM: risultati C0-C4 delle nuove suite automatiche (N=12).

Appendice H - Reliability e tracciabilità delle evidenze

Classe	N manuale finale	Prob. per test	PASS	FAIL	R_hat	Q_hat
PCEnhancer	40	0.025000000000	40	0	1.000000	0.000000
ListIteratorWrapper	19	0.052631578947	19	0	1.000000	0.000000

Tabella H.1 - Reliability empirica sotto profilo operativo uniforme.

Area	Documentazione / evidenza principale
PCEnhancer T_BB	isw2/docs/testing/pcenhancer-black-box.md
PCEnhancer T_CF	isw2/docs/testing/pcenhancer-control-flow.md
PCEnhancer T_MT	isw2/docs/testing/pcenhancer-mutation-testing.md
PCEnhancer automatiche	isw2/docs/testing/pcenhancer-{random,evosuite,llm}-testing.md
ListIteratorWrapper T_BB	isw2/docs/testing/list-iterator-wrapper-black-box.md
ListIteratorWrapper T_CF	isw2/docs/testing/list-iterator-wrapper-control-flow.md
ListIteratorWrapper T_MT	isw2/docs/testing/list-iterator-wrapper-mutation-testing.md
ListIteratorWrapper automatiche	isw2/docs/testing/list-iterator-wrapper-{random,evosuite,llm}-testing.md
Refactoring PCEnhancer	isw2/docs/m4/pcenhancer-m4-analysis.md
Refactoring ListIteratorWrapper	isw2/docs/m4/listiteratorwrapper-m4-analysis.md
Confronto automatico PC C0-C4	isw2/results/testing/automatic-suite-quality/pcenhancer/automatic-suite-comparison-c0-c4.md
Confronto automatico LIW C0-C4	isw2/results/testing/automatic-suite-quality/list-iterator-wrapper/automatic-suite-comparison-c0-c4.md
Reliability	isw2/results/testing/{pcenhancer,list-iterator-wrapper}/reliability/
Classi considerate	isw2/classes.txt
Audit finale	isw2/results/testing/final-audit/

Tabella H.2 - Mappa sintetica degli artefatti del repository.